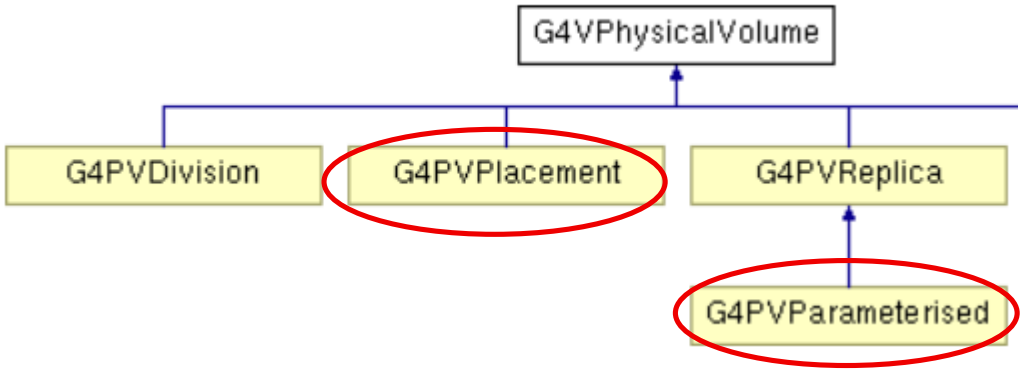
Физический объем

- Строится на основе логического объема
- Описывает положение объема в пространстве
- Позволяет одновременно описать серию одинаковых объемов или параметризовать свойства объема в зависимости от номера копии



G4PVPlacement = один объем

Единственная копия данного объема размещается в материнском объеме

G4PVParameterised = одновременное описание нескольких объемов

Возможна параметризация формы, размеров, материала, положения и поворотов в пространстве в зависимости от номера копии

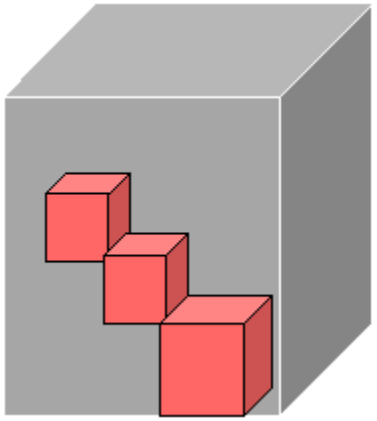
G4AssemblyVolume

одновременно размещается несколько не вложенных друг в друга объемов, которые ведут себя как единое целое при геометрических преобразованиях (поворотах и т.д.)

Параметризация физического объема

Параметризованные объёмы – это возможность размещения копий объёмов, которые могут различаться по размерам, форме или материалам.

Пользователь должен определить материнский объём, в котором будут располагаться параметризованные объёмы.



Мир **не может** быть параметеризован.

Материнский объём должен быть определён логическим или физическим.

Форма, размер, материал и положение копий в материнском объёме параметризуются как функция номера копии.

Пользователь должен написать свой класс – наследник G4VPVParameterisation и определить в нём необходимые свойства объёма.

Заданная пользователем параметризация используется при создании физических объёмов во время выполнения программы.

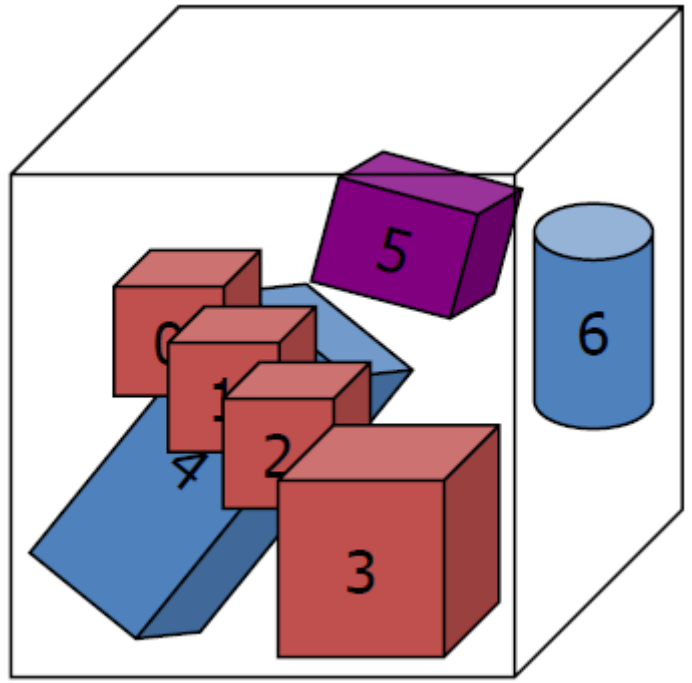
Параметризация физического объема

```
G4PVParameterised(const G4String& pName,
                  G4LogicalVolume* pLogical,
                  G4LogicalVolume* pMother,
                  const EAxis pAxis,
                  const G4int nReplicas,
                  G4VPVParameterisation* pParam
                  G4bool pSurfChk=false);
```

Конструктор физического объёма с параметризованными копиями

Одномерная параметризация возможна вдоль одной из координатных осей X, Y, Z (kXAxis, kYAxis, kZAxis).
Трёхмерная параметризация задаётся при kUndefined.

Копии размещённых объёмов не должны пересекаться с материнским объёмом и друг с другом



Параметризация физического объема

Ограничения:

- Можно параметризовать только простые формы

Положение копии в материнском объёме (чисто виртуальный метод)

Форма

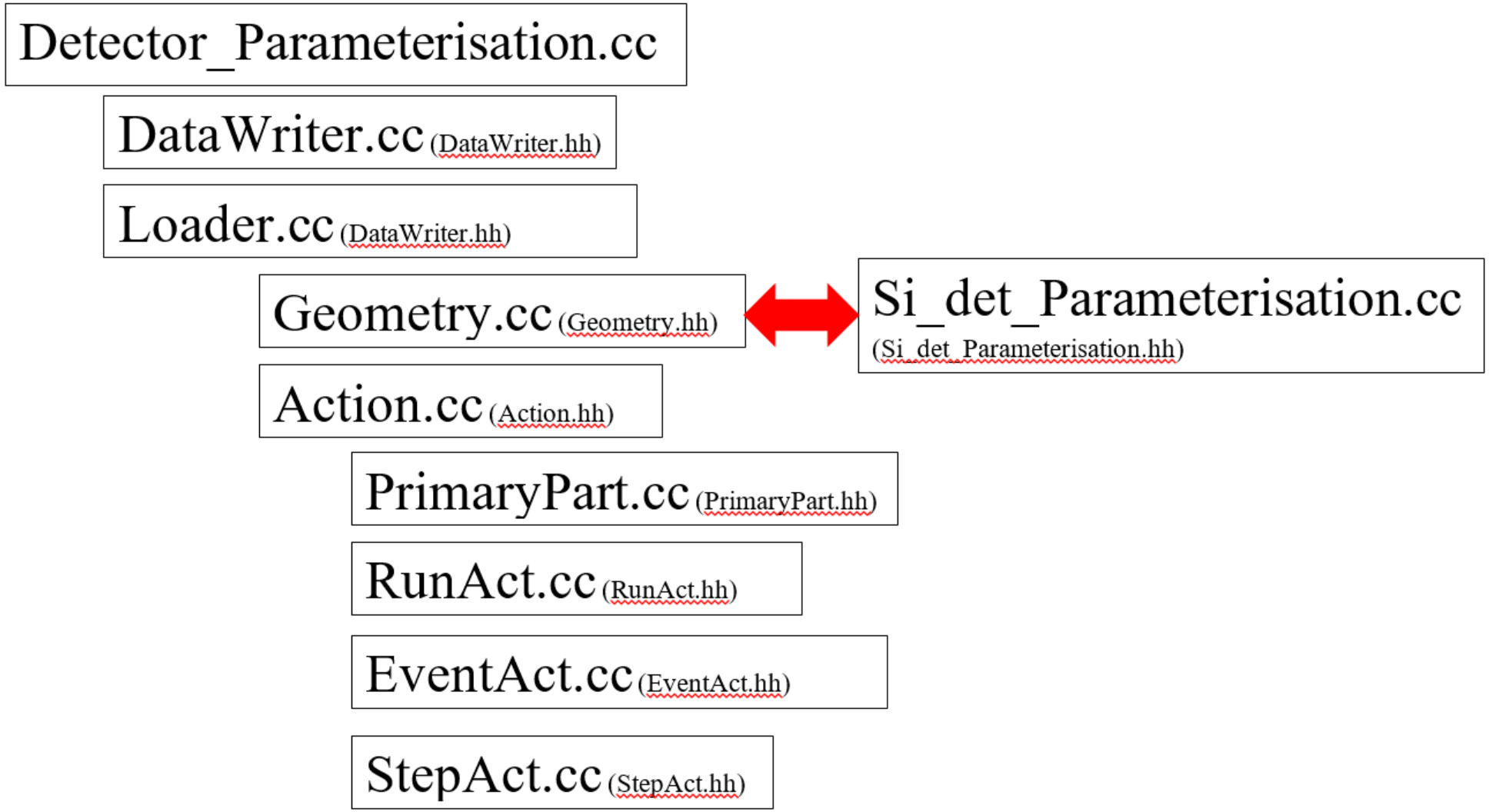
Материал

Размер

Public Member Functions

	G4VPVParameterisation ()
virtual	**~G4VPVParameterisation** ()
virtual void	**ComputeTransformation** (const **G4int**, **G4VPhysicalVolume** *) const =0
virtual **G4VSolid** *	**ComputeSolid** (const **G4int**, **G4VPhysicalVolume** *)
virtual **G4Material** *	**ComputeMaterial** (const **G4int** repNo, **G4VPhysicalVolume** *currentVol, const **G4VTouchable** *parentTouch=0)
virtual **G4bool**	**IsNested** () const
virtual **G4VVolumeMaterialScanner** *	**GetMaterialScanner** ()
virtual void	**ComputeDimensions** (**G4Box** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Tubs** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Trd** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Trap** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Cons** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Sphere** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Orb** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Torus** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Para** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Polycone** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Polyhedra** &, const **G4int**, const **G4VPhysicalVolume** *) const
virtual void	**ComputeDimensions** (**G4Hype** &, const **G4int**, const **G4VPhysicalVolume** *) const

проект Detector_Parameterisation



Параметризация физического объема

ПРОЕКТ “Detector_Parameterisation”.

Материнский объём

```
Si_box  = new G4Box("Si_box", 10. * cm, 1. * cm, 1. * cm);
Si_log  = new G4LogicalVolume(Si_box, world_mat,"Si_log");
```

Параметризуемый объём

```
Si_det_box = new G4Box(«Si_det_box", 1. * cm, 1. * cm, 1. * cm);
Si_det_log = new G4LogicalVolume(Si_det_box, Si_mat, "Si_det_log");
```

Пользовательский класс с методами:
ComputeDimensions, ComputeTransformation, ComputeSolid, ComputeMaterial

```
G4VPVParameterisation* Si_det = new Si_det_Parameterisation();
```

Размещение параметризованных копий объёма в материнском объёме

```
Si_det_pvpl= new G4PVParameterised(“Si_det_pvpl", Si_det_log, Si_log, kXAxis, 10, Si_det);
```

Параметризация физического объема

Параметризация положения копий в материнском объёме.

```
void Si_det_Parameterisation::ComputeTransformation (const G4int copyNo, G4VPhysicalVolume* physVol) const
{
 G4double Xposition= 0.* cm;
 G4double Yposition= 0.* cm;
 G4double Zposition= 0.* cm;
 Xposition = -9.*cm + (2.*cm)*copyNo;
 G4ThreeVector origin(Xposition, Yposition, Zposition);
 physVol->SetTranslation(origin);
 physVol->SetRotation(0);
}
```

Параметризация размера.

```
void Si_det_Parameterisation::ComputeDimensions (G4Box& Si_det_box, const G4int, const G4VPhysicalVolume*) const
{
 G4double XhalfLength = 0.5* cm;
 G4double YhalfLength = 1 * cm;
 G4double ZhalfLength = 1.* cm;
 Si_det_box.SetXHalfLength(XhalfLength);
 Si_det_box.SetYHalfLength(YhalfLength);
 Si_det_box.SetZHalfLength(ZhalfLength);
}
```

G4AssemblyVolume (контейнеры для логических объёмов)

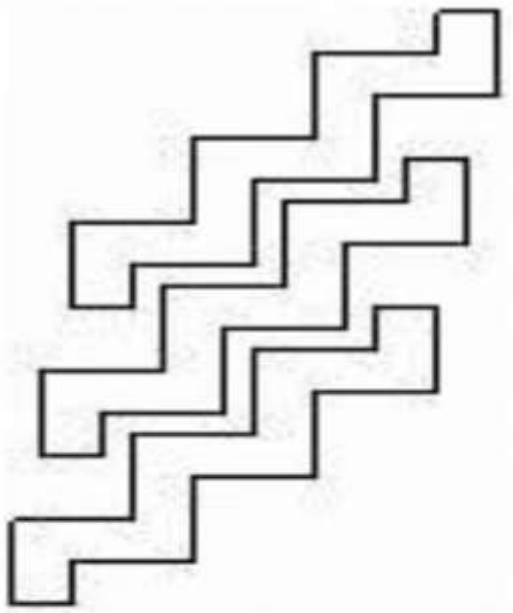
При использовании повторяющихся блоков той или иной конструкции можно сгруппировать такие объемы в общий контейнер, представленный в Geant4 классом **G4AssemblyVolume**.

Контейнер позволяет группировать логические объемы с целью многократного воспроизведения в мире.

Класс **G4AssemblyVolume** имеет два конструктора:

Параметризованный

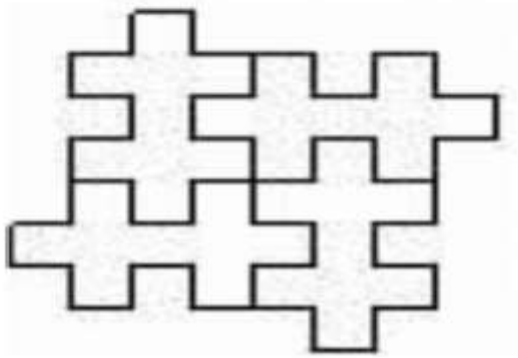
```
G4AssemblyVolume( G4LogicalVolume* volume,        //Логический объем, центр которого
                                                  //станет началом координат
                  G4ThreeVector& translation,     //Вектор смещения
                  G4RotationMatrix* rotation)     //Матрица поворота
```



И конструктор «по умолчанию» **G4AssemblyVolume()**

В данном случае создается пустой контейнер, а после добавления первого логического объема, центр данного объема станет началом координат.

В случае применения параметризованного конструктора смещение и поворот осуществляются относительно центра контейнера.



G4AssemblyVolume

Существует метод на добавление логического объема в уже существующий контейнер.

```
void AddPlacedVolume( G4LogicalVolume* pPlacedVolume,      //размещаемый объем
                      G4ThreeVector& translation,          //смещение
                      G4RotationMatrix* rotation);         //поворот
```

При многократном добавлении одного и того же логического объема в контейнер новые логические копии этого объема не появляются. Однако, все размещенные таким образом физические объемы будут иметь уникальные имена

Для размещения контейнера в материнском мире следует воспользоваться методом:

```
void MakeImprint( G4LogicalVolume* pMotherLV,              //материнский объем
                  G4ThreeVector& translationInMother,      //смещение в мат. объеме
                  G4RotationMatrix* pRotationInMother,     //поворот в мат. объеме
                  G4int copyNumBase = 0,                   //номер копии
                  G4bool surfCheck = false );              //проверка на пересечение
```

G4AssemblyVolume (проект AssemblyHit)

Класс → Метод

Geometry.cc → Geometry : public G4VUserDetectorConstruction → G4VPhysicalVolume* Geometry::Construct()

Создание логического объёма для размещения в контейнере логических объёмов

```
Si_mat=nist->FindOrBuildMaterial("G4_Si");
Si_det_box  = new G4Box("Si_det_vol", 1. * cm, 1. * cm, 1. * cm);
Si_det_log1  = new G4LogicalVolume(Si_det_box, Si_mat,"Si_det_vol_log1");
```

Создание пустого контейнера логических объёмов, его заполнение

```
 assemblyDetector = new G4AssemblyVolume();
G4RotationMatrix* assembly_RM = new G4RotationMatrix(0, 0, 0);
G4ThreeVector assembly_vector = G4ThreeVector(0, 0, 0.*cm);
assemblyDetector->AddPlacedVolume(Si_det_log1, assembly_vector, assembly_RM);
G4RotationMatrix* assembly_RM1 = new G4RotationMatrix(0, 0, 0);
 assembly_RM1->rotateX(45.*deg); assembly_RM1->rotateY(45.*deg); assembly_RM1->rotateZ(45.*deg);
assembly_vector.setZ(3.*cm);
assemblyDetector->AddPlacedVolume(Si_det_log1, assembly_vector, assembly_RM1);
G4RotationMatrix* assembly_RM2 = new G4RotationMatrix(0, 0, 0);
assembly_RM2->rotateX(10.*deg); assembly_RM2->rotateY(20.*deg); assembly_RM2->rotateZ(30.*deg);
assembly_vector.setZ(6.*cm);
assemblyDetector->AddPlacedVolume(Si_det_log1, assembly_vector, assembly_RM2);
```

G4AssemblyVolume (проект AssemblyHit)

Размещение заполненного контейнера логических объёмов в материнском объёме

```
G4RotationMatrix* assembly_RM = new G4RotationMatrix(0, 0, 0);
G4ThreeVector assembly_vector = G4ThreeVector(0, 0, 0.*cm);
assembly_vector.setZ(5.*cm);
 assemblyDetector->MakeImprint(world_log, assembly_vector, assembly_RM);
```

Физические объёмы генерируются при вызове метода **MakeImprint**.
Имена объёмов генерируются в следующем формате:

```
av_WWW_impr_XXX_YYY_ZZZ
```

WWW – номер контейнера
XXX – номер размещения контейнера
YYY – имя размещаемого логического объёма
ZZZ – индекс логического объёма в контейнере (порядковый номер заполнения контейнера)